API base class

class **mini.apis.base_api.MiniApiResultType**(*value*) [source]

Bases: `enum.Enum`

Api return result status type

Success: Received robot reply

Timeout: Reply to the robot received within the timeout period

Unsupported: The instruction is not supported

> **Success**= *1*

> **Timeout**= *2*

> **Unsupported**= *3*

class **mini.apis.base_api.BaseApi** [source]

Bases: `abc.ABC`

Message api base class

> *async* **send**(*cmd_id: int*, *message*, *timeout: int*)→ Union[object, bool] [source]
>
> Send message method
>
> Note: Called internally by subclass functions, subclass instances cannot be called.
>
> **Parameters:**
> - **cmd_id** (*int*) – Supported command id, for example: mini.apis.cmdid.PLAY_ACTION_REQUEST
> - **message** (*Message*) – Supported message entity, for example: mini.pb2.PlayActionRequest
> - **timeout** (*int*) – timeout time. When timeout<=0, it means that there is no need to wait for the robot to reply.When timeout>0, it means that it needs to wait for the robot to reply.
>
> **Returns:**
> If the instruction is not supported, return tuple(MiniApiResultType.Unsupported,None)
>
> If the instruction is supported

When timeout<=0, bool is returned, indicating whether the message was sent successfully

When timeout>0

If the message times out or fails to be sent, tuple(MiniApiResultType.Timeout,None) is returned

If the message has a reply, it will return tuple(MiniApiResultType.Success,result), result is the corresponding reply message

> *async* **execute**() [source]

Send instructions

After serializing the supported message, write it to the socket Implemented by subclasses

---

class **mini.apis.base_api.BaseApiNeedResponse** [source]

Bases: `mini.apis.base_api.BaseApi` , `abc.ABC`

The message api base class that needs to be replied, timeout cannot be empty

> *async* **send**(*cmd_id*, *data*, *timeout: int*) [source]

Override the parent method

Check timeout, must be >0

---

class **mini.apis.base_api.BaseApiNoNeedResponse** [source]

Bases: `mini.apis.base_api.BaseApi` , `abc.ABC`

Message api base class without reply

> *async* **send**(*cmd_id*, *message*, *timeout: int = 0*) [source]

Override the parent method

Set timeout to 0

---

class **mini.apis.base_api.BaseEventApi**(*cmd_id: int*, *message*, *is_repeat: bool = True*, *timeout: int = 0*, *handler: Optional[Callable[[...], None]] = None*) [source]

Bases: `mini.apis.base_api.BaseApiNoNeedResponse` , `mini.channels.websocket_client.AbstractMsgHandler` , `abc.ABC`

Event class message api base class

When the event handler is registered, the event message actively pushed by the robot

Parameters:

- **cmd_id** (*int*) – registered instruction id
- **message** (*Message*) – the registration message sent to the robot
- **is_repeat** (*bool*) – Whether to repeat the monitoring, the default is True
- **timeout** (*int*) – timeout time, the default is 0
- **handler** (*Callable*) – event message handler, f(message)

`set_handler`(*handler: Optional[Callable[[...], None]] = None*) [source]

Set up event message handler

Parameters: **handler** (*Callable*) – event message handler, f(message)

`start`() [source]

Start the listener

`stop`() [source]

Stop listener

The subclass needs to notify the robot to stop reporting the event

`handle_msg`(*message*) [source]